[Huyen Nguyen]IPC: Pipes, FIFOs, Message Queues, Shared Memory, Signals

1. Pipes

1.1. Parent-Child Process Communication via Pipe

The parent calls pipe() -> the parent holds both filedes[0] (read end) and filedes[1] (write end).

The parent calls fork() -> the child inherits the two file descriptors from the parent.

Close the unused file descriptor ends.

Why must you close unused ends?

```
int main()
{
    int fd[2];
    pipe(fd);
    if (fork() == 0)
    {
        char buffer[100];
        int bytes_read;
        while ((bytes_read = read(fd[0], buffer, sizeof(buffer) - 1)) > 0)
        {
            buffer[bytes_read] = '\0';
            printf("[Child] Received: %s\n", buffer);
        }
        close(fd[0]);
        return 0;
    }
```

```
    close(fd[0]);
    printf("[Parent] Sending message 1...\n");
    write(fd[1], "Message 1", 9);
    sleep(1);
    printf("[Parent] Sending message 2...\n");
    write(fd[1], "Message 2", 9);
    sleep(1);
    printf("[Parent] Done sending. Closing write end fd[1].\n");
    close(fd[1]);
    wait(NULL);
    printf("[Parent] Program finished successfully.\n");
    return 0;
}
```

Output

```
[Parent] Sending message 1...
[Child] Received: Message 1
[Parent] Sending message 2...
[Child] Received: Message 2
[Parent] Done sending. Closing write end fd[1]./*Deadlock*/
```

-> If the child process does not close its write end, then after the parent finishes writing and exits, any subsequent read call by the child will deadlock because the operating system assumes someone might still write more data.

->The child process will only receive the EOF (End-Of-File, returning 0) signal once all write ends are closed.

Why can't a single pipe be used for bi-directional communication?

```
int main()
{
    int fd[2];
    pipe(fd);
    if (fork() == 0)
    {
        close(fd[1]);
        char c;
        while (read(fd[0], &c, 1) > 0)
        {
```

```
            printf("[Child 1] grabbed: %c\n", c);
            usleep(10000);
        }
        close(fd[0]);
        return 0;
    }
    if (fork() == 0)
    {
        close(fd[1]);
        char c;
        while (read(fd[0], &c, 1) > 0)
        {
            printf("[Child 2] grabbed: %c\n", c);
            usleep(10000);
        }
        close(fd[0]);
        return 0;
    }
    close(fd[0]);
    char text[] = "ABCDEFGH";
    printf("[Parent] Writing string \"%s\" into the pipe...\n\n", text);
    write(fd[1], text, sizeof(text) - 1);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Output

```
[Parent] Writing string "ABCDEFGH" into the pipe...

[Child 1] grabbed: A
[Child 2] grabbed: B
[Child 1] grabbed: C
[Child 2] grabbed: D
[Child 1] grabbed: E
[Child 2] grabbed: F
[Child 2] grabbed: G
[Child 1] grabbed: H
```

Because when the parent writes data into the pipe, it might read back its own data before the child gets a chance to read it. Alternatively, if both processes race to read, the data gets fragmented->Race Condition.

If a process is meant to write, why must its read end be closed?

```
int main()
{
    int fd[2];
    pipe(fd);
    if (fork() == 0)
    {
        close(fd[1]);
        char buf[10];
       int n = read(fd[0], buf, 5);
        printf("[Child] Read %d bytes\n", n);
        close(fd[0]);
        exit(0);
    }
    sleep(1);
    char chunk[1024];
    int total = 0;
    while (1)
    {
        write(fd[1], chunk, sizeof(chunk));
        total += 1;
        printf("[Parent] Written: %d KB\n", total);
    }
    return 0;
}
```

Output

```
[Parent] Written: 1 KB
[Child] Read 5 bytes
[Parent] Written: 2 KB
[Parent] Written: 3 KB
...
[Parent] Written: 63 KB
[Parent] Written: 64 KB
```

If a process writes to a pipe that has no remaining read ends, the Kernel sends a SIGPIPE signal to terminate the writer process.

If the writer process does not close its own read end, the Kernel will never send the SIGPIPE signal-> the process continues writing indefinitely -> the pipe buffer becomes full-> the write() function blocks.

1.2. Pipes allow communication between related processes

A common ancestor process must create the pipe, and subsequent child processes inherit it -> allowing the processes to communicate with one another.

Two siblings

The parent creates two child processes -> the parent closes both ends of the pipe.

The two newly created child processes can still communicate with each other.

```
ls -l | wc -l
19
```

The Bash shell process is the parent process that creates the pipe.

- Child 1 runs the ls -l command (writes output into the pipe).
- Child 2 runs the wc -l command (reads output from the pipe to count lines).

1.3. Using a Pipe for Synchronization Between Parent and Child Processes

Idea : The parent process waits for all child processes to complete their work before moving on to the next step.

```
int main()
{
    int sync_pipe[2];
    pipe(sync_pipe);
    for (int i = 1; i <= 3; i++)
    {
        if (fork() == 0)
        {
            close(sync_pipe[0]);
            printf("[Child %d] Working on task (takes %d seconds)...\n", i, i);
            sleep(i);
```

```
            printf("[Child %d] Done! Closing write end.\n", i);
            close(sync_pipe[1]);
            exit(0);
        }
    }
    close(sync_pipe[1]);
    char dummy;
    read(sync_pipe[0], &dummy, 1);
    close(sync_pipe[0]);
    while (wait(NULL) > 0);
    return 0;
}
```

Output

```
[Child 1] Working on task
[Child 3] Working on task
[Child 2] Working on task
[Child 1] Closing write end.
[Child 2] Closing write end.
[Child 3] Closing write end.
```

Solution :

- The parent creates a pipe -> calls fork() to spawn 3 child processes -> there are now 4 write ends open in total.
- The parent closes its own write end -> executes read(pfd[0], &dummy, 1); -> goes to sleep while waiting for the 3 children to finish.
- Whenever a child process finishes its task, it closes its write end -> once no write ends remain open, the parent wakes up as read() returns 0 (EOF) -> the parent resumes execution.

Why use a pipe instead of signals?

Signals are not queued: If two child processes finish at the exact same time and send a signal simultaneously, Linux will coalesce (merge) the two signals into a single signal -> the parent miscounts the completed processes.

1.4. Coprocess in Linux Shell

Background processes are difficult to interact with bidirectionally, as standard pipelines only pass data unidirectionally from cmd1 to cmd2.

-> A coprocess is a background process running concurrently with the shell, where the shell automatically connects two pipes to that process.

Syntax

```
Named: coproc MY_PROC { command; }


Unnamed: coproc command args
```

```
huyen@huyen:~$ coproc CALC { bc; } /* The parent process (Shell) spawns a child process `bc` running in the background as a coprocess, while creating two pipes to connect the parent and child */
[1] 10196
huyen@huyen:~$ echo "15 * 4" >&"${CALC[1]}" /* Shell sends data to `bc` */
huyen@huyen:~$ read -u "${CALC[0]}" result /* `bc` sends output for the shell to receive */
huyen@huyen:~$ echo "Result is: $result"
Result is: 60
huyen@huyen:~$ exec {CALC[1]}>&- /* Shell closes the write end */
```

-> coproc is used to create a background service process for bidirectional interaction via pipes without requiring temporary files.

1.5. Why is popen() needed?

To run a Shell command inside code and capture its standard output, must:

- pipe(): create the pipe
- fork(): create the child process
- dup2(): redirect stdout/stdin
- exec(): execute the shell
- Close redundant file descriptors

-> popen() encapsulates all 5 steps above.

Without popen()

```
int main()
{
    int pfd[2];
    pipe(pfd);
    pid_t pid = fork();
    if (pid == 0)
```

```
    {
        close(pfd[0]);
        dup2(pfd[1], STDOUT_FILENO);
        close(pfd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("execlp failed");
        exit(1);
    }
    close(pfd[1]);
    FILE *fp = fdopen(pfd[0], "r");
    char line[256];
    while (fgets(line, sizeof(line), fp) != NULL)
    {
        printf("[Without popen] %s", line);
    }
    fclose(fp);
    int status;
    waitpid(pid, &status, 0);
    return 0;
}
```

With popen()

```
int main()
{
    FILE *fp = popen("ls -l", "r");
    if (fp == NULL)
    {
        return 1;
    }
    char line[256];
    while (fgets(line, sizeof(line), fp) != NULL)
    {
        printf("[With popen]    %s", line);
    }
    pclose(fp);
    return 0;
}
```

Why must pclose() be used instead of fclose()?

- fclose() only closes the pipe stream -> the child process running the underlying shell becomes a zombie process.
- pclose(): closes the pipe stream and automatically calls waitpid() to reap the child process.

2. FIFOs

2.1. How does a FIFO differ from a pipe()?

A pipe is an anonymous pipe that exists solely in RAM; only processes with a common ancestor (a parent-child relationship) can inherit and share it.

A FIFO is also a pipe located in RAM, but it is given a specific name and attached to the filesystem hierarchy. Any process can communicate with another via this file path.

A FIFO file shows a size of 0 bytes because the data is buffered directly in RAM rather than saved to disk.

2.2. Synchronous Handshake

Both the reading and writing processes must join at the pipe before either can proceed with execution.

- Terminal 1

```
mkfifo test_fifo
cat test_fifo
/* Blocks until a process opens it for writing */
Hi FIFO
```

- Terminal 2

```
echo "Hi FIFO" > test_fifo
```

-> Both the writer and reader must be present at the pipe for data transfer to begin.

2.3. Combining FIFO with tee for Pipeline Branching

A single data stream can be delivered to two (or more) processes for concurrent processing using tee + FIFO.

```
mkfifo myfifo /* Creates a named pipe */
wc -l < myfifo & /* Runs a background process reading from the pipe */
```

```
ls -l | tee myfifo | sort -k5n /* Writes data through the pipeline */
```

Output

```
prw-rw-r-- 1 huyen huyen     0 Sep 15 13:39 test_fifo
prw-rw-r-- 1 huyen huyen     0 Sep 15 14:00 myfifo
total 288
-rw-r--r-- 1 huyen huyen   284 Sep 15 11:40 compare_with_popen.c
-rw-r--r-- 1 huyen huyen   516 Sep 15 10:33 pipe_writer_forgot_read.c
...
-rwxrwxr-x 1 huyen huyen 16392 Sep 11 16:46 pipe_demo
-rwxrwxr-x 1 huyen huyen 16472 Sep 15 11:42 compare_without_popen
```

-> Prints both the line count of the output and the detailed list sorted by file size in ascending order.

Explanation: tee myfifo creates two identical copies of the stream:

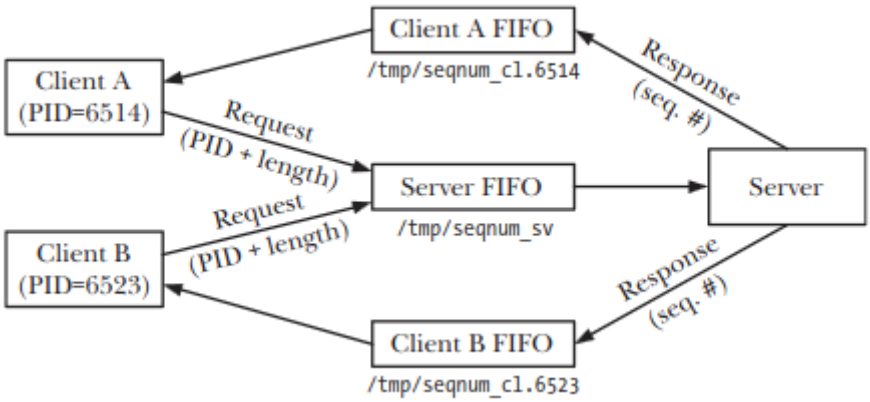
- Branch 1 (side stream): Pumps data directly into myfifo, where the background wc -l counts the lines.
- Branch 2 (main stream): Forwards data through the standard pipe to sort -k5n.

2.4. Client - Server Model

Multiple applications share and access a single resource concurrently without conflicts -> Client-Server Model.

Server: Holds the resource and runs continuously.

Client: Sends a request to the Server whenever needed -> Receives the returned result.



Request Direction (Left to Right): The Client sends a request containing its PID and the number of sequence numbers it wants the Server to allocate-> The request is forwarded to the Server.

Response Direction (Right to Left): The Server responds to each Client using a corresponding dedicated FIFO created by that specific Client.

2.5. Message Boundary Issues

When Client A and Client B send data simultaneously, how does the Server determine where Client A's message ends?

- Delimiter character

```
write(fd, "HELLO\n", 6);
write(fd, "HI\n", 3);
write(fd, "BYE\n", 4);
```

- Length-Prefixed Headers

```
[5] HELLO [2] HI [3] BYE
 ^         ^       ^
(Header) (Header) (Header)
```

- Fixed-Size Messages: Every packet has an identical fixed length; padding is inserted if the actual data is shorter than the required size.

2.6. Non-blocking I/O

While waiting for one end of the FIFO to open, a program can perform other tasks in the meantime -> O_NONBLOCK flag.

Why can O_NONBLOCK be used when opening for reading, but causes issues when opening for writing?

For the reader: If no writer is connected yet, reading simply returns 0 bytes or fails with EAGAIN -> no fatal errors occur.

For the writer: Attempting to write to a pipe without any reader connected causes the Operating System to trigger a SIGPIPE signal, terminating the process.

What is O_NONBLOCK used for?

```
[ Process X ]                                [ Process Y ]
Step 1: Open FIFO A for READING              Step 1: Open FIFO B for READING
        (Blocks waiting for FIFO A writer...)        (Blocks waiting for FIFO B writer...)
Step 2: Open FIFO B for WRITING              Step 2: Open FIFO A for WRITING
```

-> Both processes become permanently trapped at Step 1.

-> Using the O_NONBLOCK flag allows both processes to bypass Step 1 without blocking, enabling them to proceed to Step 2 and open the write ends for each other.

The "Side Effect" of the O_NONBLOCK Flag

When using the O_NONBLOCK flag during the open() call -> this flag is persisted in the file descriptor status -> subsequent read() and write() calls automatically inherit the non-blocking behavior.

Solution: Use the fcntl() function as an "On/Off Switch" to modify file descriptor status flags dynamically:

```
flags = fcntl(fd, F_GETFL);     /*  Retrieve current flags*/
flags &= ~O_NONBLOCK;          /*  Use Bitwise AND-NOT (&~) to CLEAR the O_NONBLOCK bit*/
fcntl(fd, F_SETFL, flags);      /* Apply updated flags back to the system*/
```

Lab 1 : a chat program between two processes over FIFO

Process 1

```
#define FIFO_1 "/tmp/fifo_1_to_2"
#define FIFO_2 "/tmp/fifo_2_to_1"
int main()
{
    mkfifo(FIFO_1, 0666);
    mkfifo(FIFO_2, 0666);
    int fd_write = open(FIFO_1, O_WRONLY);
    int fd_read  = open(FIFO_2, O_RDONLY);
    printf("[User 1] Connected\n\n");
    char send_buf[256], recv_buf[256];
    while (1)
    {
        printf("User 1: ");
        fflush(stdout);
        if (fgets(send_buf, sizeof(send_buf), stdin) == NULL) break;
        write(fd_write, send_buf, strlen(send_buf) + 1);
        if (strncmp(send_buf, "exit", 4) == 0) break;
        if (read(fd_read, recv_buf, sizeof(recv_buf)) <= 0) break;
        if (strncmp(recv_buf, "exit", 4) == 0)
```

```
        {
            printf("User 1\n");
            break;
        }
        printf("User 1: %s", recv_buf);
    }
    close(fd_write);
    close(fd_read);
    unlink(FIFO_1);
    unlink(FIFO_2);
    return 0;
}
```

Process 2

```
#define FIFO_1 "/tmp/fifo_1_to_2"
#define FIFO_2 "/tmp/fifo_2_to_1"
int main()
{
    mkfifo(FIFO_1, 0666);
    mkfifo(FIFO_2, 0666);
    printf("[User 2] Connecting to User 1...\n");
    int fd_read  = open(FIFO_1, O_RDONLY);
    int fd_write = open(FIFO_2, O_WRONLY);
    printf("[User 2] Connected\n\n");
    char send_buf[256], recv_buf[256];
    while (1)
    {
        if (read(fd_read, recv_buf, sizeof(recv_buf)) <= 0) break;
        if (strncmp(recv_buf, "exit", 4) == 0)
        {
            printf("User 2\n");
            break;
        }
        printf("User 2: %s", recv_buf);
        fflush(stdout);
        if (fgets(send_buf, sizeof(send_buf), stdin) == NULL) break;
        write(fd_write, send_buf, strlen(send_buf) + 1);
        if (strncmp(send_buf, "exit", 4) == 0) break;
    }
```

```
    close(fd_read);
    close(fd_write);
    unlink(FIFO_1);
    unlink(FIFO_2);
    return 0;
}
```

Outout

Process 1

```
[User 1] Connected

User 1: From 1
User 1: From 2
User 1:
```

Process 2

```
[User 2] Connecting to User 1...
[User 2] Connected

User 2: From 1
From 2
User 2: From 1
```

Because FIFOs only support one-way (half-duplex) transmission, enabling two-way chat strictly requires using two separate FIFO files:

- FIFO_1 (/tmp/fifo_1_to_2): Dedicated for User 1 sending -> User 2 receiving.
- FIFO_2 (/tmp/fifo_2_to_1): Dedicated for User 2 sending -> User 1 receiving.

3. System V IPC

3.1. IPC Identifiers and Keys

How the get() functions work: "Exchange a Key for an ID"

The msgget(), semget(), and shmget() functions operate similarly to the open() function:

- Take a key and flags as inputs

- If it doesn't exist -> Create a new one and return the ID number
- If it already exists -> Open it and return the ID

Send

```
struct message
{
    long mtype;
    char mtext[100];
};
int main()
{
    key_t key = 0x1234;
    int msg_id = msgget(key, IPC_CREAT | 0666);
    if (msg_id == -1)
    {
        perror("msgget");
        exit(1);
    }
    printf("[Sender] Key 0x%x -> ID: %d\n", key, msg_id);
    struct message msg;
    msg.mtype = 1;
    strcpy(msg.mtext, "Message from an unrelated process");
    msgsnd(msg_id, &msg, strlen(msg.mtext) + 1, 0);
    printf("[Sender] Sent: \"%s\"\n", msg.mtext);
    return 0;
}
```

Receive

```
struct message
{
    long mtype;
    char mtext[100];
};
int main()
{
    key_t key = 0x1234;
    int msg_id = msgget(key, 0);
    if (msg_id == -1)
    {
```

```
        perror("msgget");
        exit(1);
    }
    printf("[Receiver] Key 0x%x -> ID: %d\n", key, msg_id);
    struct message msg;
    msgrcv(msg_id, &msg, sizeof(msg.mtext), 1, 0);
    printf("[Receiver] Received: \"%s\"\n", msg.mtext);
    msgctl(msg_id, IPC_RMID, NULL);
    printf("[Receiver] Queue deleted.\n");
    return 0;
}
```

Output - Send

```
[Sender] Key 0x1234 -> ID: 5
[Sender] Sent: "Message from an unrelated process!"
```

Message Queues

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x00001234 5          huyen      666        35           1
```

Output - Receive

```
[Receiver] Key 0x1234 -> ID: 5
[Receiver] Received: "Message from an unrelated process!"
[Receiver] Queue deleted.
```

3.2. Permission Structure

Who is allowed to send messages, read data, or remove IPC objects?

Inside the struct ipc_perm structure:

```
struct ipc_perm {
    uid_t  cuid;  /* Creator User ID: The original creator (fixed)*/
    gid_t  cgid;  /* Creator Group ID: The original creator's group*/

    uid_t  uid;   /* Owner User ID: The current owner (can be changed)*/
    gid_t  gid;   /* Owner Group ID: The current owner's group*/
```

```
};
```

- Creator (cuid): The process that initially created the object (by calling functions for Message Queues, Shared Memory, etc.).
- Owner (uid): The current owner of the object.

```
/* Call IPC_STAT to copy ownership details from the Kernel into the shmds variable*/
shmctl(id, IPC_STAT, &shmds);

/* Assign newuid*/
shmds.shm_perm.uid = newuid;

/* Call IPC_SET to pass the updated ownership details back to the Kernel*/
shmctl(id, IPC_SET, &shmds);
```

Kernel permission evaluation order:

- Root: Has full administrative privileges (highest priority)
- Owner / Creator: Applies the Owner permission bits
- Same Group: Applies the Group permission bits
- Others: Applies the Other permission bits

-> Only the current owner (uid), the original creator (cuid), or the root account has permission to remove an IPC object

3.3. IPC Limits

Why does the Kernel enforce limits on IPC? -> IPC structures reside directly within Kernel RAM. Without limits, an infinite loop bug could consume all available RAM, causing the entire system to crash.

```
struct big_message
{
    long mtype;
    char mtext[10000];
};
struct normal_message
{
    long mtype;
    char mtext[4096];
};
int main()
{
```

```
    key_t key = 0x9999;
    int msqid = msgget(key, IPC_CREAT | 0666);
    if (msqid == -1)
    {
        perror("msgget");
        exit(1);
    }
    printf("[Case 1: MSGMAX 8KB limit]\n");
    struct big_message msg_too_big;
    msg_too_big.mtype = 1;
    memset(msg_too_big.mtext, 'A', sizeof(msg_too_big.mtext));
    if (msgsnd(msqid, &msg_too_big, sizeof(msg_too_big.mtext), 0) == -1)
    {
        printf("Sent 10KB -> %s\n\n", strerror(errno));
    }
    printf("[Case 2: MSGMNB 16KB queue limit]\n");
    struct normal_message msg;
    msg.mtype = 1;
    memset(msg.mtext, 'B', sizeof(msg.mtext));
    int count = 0;
    while (1)
    {
        if (msgsnd(msqid, &msg, sizeof(msg.mtext), IPC_NOWAIT) == -1)
        {
            printf("Message #%d (4KB) -> %s\n", count + 1, strerror(errno));
            break;
        }
        count++;
        printf("Message #%d (4KB) -> Sent (Queue: %d / 16384 bytes)\n", count, count * 4096);
    }
    msgctl(msqid, IPC_RMID, NULL);
    return 0;
}
```

Output

```
[Case 1: MSGMAX 8KB limit]
Sent 10KB -> Invalid argument

[Case 2: MSGMNB 16KB queue limit]
```

```
Message #1 (4KB) -> Sent (Queue: 4096 / 16384 bytes)
Message #2 (4KB) -> Sent (Queue: 8192 / 16384 bytes)
Message #3 (4KB) -> Sent (Queue: 12288 / 16384 bytes)
Message #4 (4KB) -> Sent (Queue: 16384 / 16384 bytes)
Message #5 (4KB) -> Resource temporarily unavailable
```

Case 1. Sending a message that exceeds the maximum message size

- Attempting to send a 10,000-byte message when MSGMAX allows a maximum of 8,192 bytes -> Returns error EINVAL (Invalid argument)

Case 2. Exceeding the total queue capacity

- Messages of 4 KB each are sent successfully, filling the queue: 4 KB -> 8 KB -> 12 KB -> 16 KB
- Thanks to the IPC_NOWAIT flag, the function does not block/hang and instead returns error EAGAIN (Resource temporarily unavailable)

Practical Limits

```
------ Messages Limits --------
max queues system wide = 32000              /* Maximum of 32,000 message queues system-wide */
max size of message (bytes) = 8192          /* Maximum size of a single message */
default max size of queue (bytes) = 16384   /* Total capacity of pending messages in a queue
*/

------ Shared Memory Limits --------
max number of segments = 4096               /* Maximum number of shared memory segments */
max seg size (kbytes) = 18014398509465599   /* Maximum size of a single segment (~16 Exabytes)
*/
max total shared memory (kbytes) = 18446744073709551612
min seg size (bytes) = 1                    /* Minimum segment size */

------ Semaphore Limits --------
max number of arrays = 32000                /* Maximum number of semaphore arrays */
max semaphores per array = 32000            /* Maximum semaphores per array */
max semaphores system wide = 1024000000     /* Total semaphores system-wide */
max ops per semop call = 500                /* Maximum operations per semop call */
semaphore max value = 32767                 /* Maximum semaphore count value */
```

Root can raise these limits if an application requires handling larger data workloads.

3.4. Kernel Persistence

Pipes / File Descriptors: Process-persistent. When the process is killed, the pipe is automatically destroyed.

System V IPC: Kernel-persistent.

- When a message queue or shared memory segment is created, it remains permanently in system RAM even if the process that created it is killed.
- It disappears only when explicitly removed or when the system reboots.

Advantages and Disadvantages

Pros: Convenient Asynchronous Communication

- Process A sends a packet to the Message Queue in RAM and exits.
- Process B, operating independently of Process A's timeline, can retrieve the packet from the Message Queue in RAM at any time.

Cons: Resource Leakage

- Risks exhausting system RAM.
- Makes it difficult to determine which process was the last to use the resource to handle cleanup.

4. Message Queues

Server side: When calling msgget(), the IPC_CREAT flag must be passed to create a new message queue.

Client side: When calling msgget(), it connects to and uses the message queue created by the Server.

What happens if the Server crashes and restarts?

- The Server process dies, but the old Message Queue and its data remain in RAM (due to Kernel persistence).
- If a new Server instance restarts and reuses the old message queue, it risks reading stale/junk messages left over by the previous Server.

-> Clean up (remove) the old queue and create a fresh one.

Automatic Kernel Protection: The new ID is always different from the old ID

The Kernel always generates a new ID number (by incrementing an internal Kernel sequence counter, seq).

4.1. Client-Server Architecture with Message Queues

Model 1: Single Shared Message Queue for Server and All Clients

(Both read and write operations use the exact same queue)

Struct

```
#define SERVER_KEY 0x1111
struct request_msg
{
    long mtype;
    pid_t client_pid;
    char text[128];
};
struct response_msg
{
    long mtype;
    char text[128];
};
#endif
```

Server

```
int qid = -1;
void cleanup(int sig)
{
    (void)sig;
    if (qid != -1)
    {
        msgctl(qid, IPC_RMID, NULL);
        printf("\n[Server] Queue deleted.\n");
    }
    exit(0);
}
int main()
{
    signal(SIGINT, cleanup);
    qid = msgget(SERVER_KEY, IPC_CREAT | 0666);
    if (qid == -1)
    {
```

```
        perror("msgget");
        exit(1);
    }
    printf("[Server] Running on queue %d...\n", qid);
    struct request_msg req;
    struct response_msg resp;
    while (1)
    {
        if (msgrcv(qid, &req, sizeof(req) - sizeof(long), 1, 0) == -1)
        {
            perror("msgrcv");
            break;
        }
        printf("[Server] Recv from PID %d: %s\n", req.client_pid, req.text);
        for (int i = 0; req.text[i]; i++)
        {
            resp.text[i] = toupper(req.text[i]);
        }
        resp.text[strlen(req.text)] = '\0';
        resp.mtype = req.client_pid;
        msgsnd(qid, &resp, sizeof(resp) - sizeof(long), 0);
        printf("[Server] Sent to PID %d: %s\n", req.client_pid, resp.text);
    }
    return 0;
}
```

Client

```
int main(int argc, char *argv[])
{
    int qid = msgget(SERVER_KEY, 0666);
    if (qid == -1)
    {
        perror("msgget");
        exit(1);
    }
    pid_t my_pid = getpid();
    struct request_msg req;
    struct response_msg resp;
    req.mtype = 1;
```

```
    req.client_pid = my_pid;
    if (argc > 1)
    {
        strncpy(req.text, argv[1], sizeof(req.text) - 1);
        req.text[sizeof(req.text) - 1] = '\0';
    }
    else
    {
        strcpy(req.text, "hello server");
    }
    printf("[Client %d] Send: %s\n", my_pid, req.text);
    msgsnd(qid, &req, sizeof(req) - sizeof(long), 0);
    msgrcv(qid, &resp, sizeof(resp) - sizeof(long), my_pid, 0);
    printf("[Client %d] Recv: %s\n", my_pid, resp.text);
    return 0;
}
```

Output

Server

```
[Server] Running on queue 4...
[Server] Recv from PID 7680: From client 1
[Server] Sent to PID 7680: FROM CLIENT 1
[Server] Recv from PID 7711: From client 2
[Server] Sent to PID 7711: FROM CLIENT 2
```

Client 1

```
[Client 7680] Send: From client 1
[Client 7680] Recv: FROM CLIENT 1
```

Client 2

```
[Client 7711] Send: From client 2
[Client 7711] Recv: FROM CLIENT 2
```

Client sends a request to the server:

- Sets mtype = 1 for messages intended for the server (using mtype = 1 prevents conflicts with process IDs, as PID = 1 is reserved for system processes).

- Includes its own PID inside the message payload sent to the server.

Server receives requests: Listens exclusively for messages with mtype = 1.

Server sends responses: Each client only retrieves messages where the message type matches its specific PID.

Pros: Simple implementation; requires creating only one system-wide message queue.

Cons:

- Risk of deadlock: High volumes of simultaneous requests can fill the queue capacity, blocking new client requests.
- Queue congestion / Starvation: If a client sends a request and crashes before reading the response, that response message remains stranded in the queue indefinitely.

Model 2: One Message Queue per Client

Server

```
int server_qid = -1;
void cleanup(int sig)
{
    (void)sig;
    if (server_qid != -1)
    {
        msgctl(server_qid, IPC_RMID, NULL);
        printf("\n[Server] Server queue deleted.\n");
    }
    exit(0);
}
int main()
{
    signal(SIGINT, cleanup);
    server_qid = msgget(SERVER_KEY, IPC_CREAT | 0666);
    if (server_qid == -1)
    {
        perror("msgget");
        exit(1);
    }
    printf("[Server] Running on server queue %d...\n", server_qid);
    struct request_msg req;
    struct response_msg resp;
```

```
    while (1)
    {
        if (msgrcv(server_qid, &req, sizeof(req) - sizeof(long), 1, 0) == -1)
        {
            perror("msgrcv");
            break;
        }
        printf("[Server] Recv from Client (Queue %d): %s\n", req.client_qid, req.text);
        for (int i = 0; req.text[i]; i++)
        {
            resp.text[i] = toupper(req.text[i]);
        }
        resp.text[strlen(req.text)] = '\0';
        resp.mtype = 1;
        if (msgsnd(req.client_qid, &resp, sizeof(resp) - sizeof(long), 0) == -1)
        {
            perror("msgsnd to client queue");
        }
        else
        {
            printf("[Server] Sent to Client (Queue %d): %s\n", req.client_qid, resp.text);
        }
    }
    return 0;
}
```

Client

```
int main(int argc, char *argv[])
{
    int server_qid = msgget(SERVER_KEY, 0666);
    if (server_qid == -1)
    {
        perror("msgget server");
        exit(1);
    }
    int my_qid = msgget(IPC_PRIVATE, IPC_CREAT | 0666);
    if (my_qid == -1)
    {
        perror("msgget client queue");
```

```
        exit(1);
    }
    struct request_msg req;
    struct response_msg resp;
    req.mtype = 1;
    req.client_qid = my_qid;
    if (argc > 1)
    {
        strncpy(req.text, argv[1], sizeof(req.text) - 1);
        req.text[sizeof(req.text) - 1] = '\0';
    }
    else
    {
        strcpy(req.text, "hello server from m2");
    }
    printf("[Client (Queue %d)] Send: %s\n", my_qid, req.text);
    msgsnd(server_qid, &req, sizeof(req) - sizeof(long), 0);
    msgrcv(my_qid, &resp, sizeof(resp) - sizeof(long), 0, 0);
    printf("[Client (Queue %d)] Recv: %s\n", my_qid, resp.text);
    msgctl(my_qid, IPC_RMID, NULL);
    return 0;
}
```

Output

```
[Server] Running on server queue 8...
[Server] Recv from Client (Queue 9): From client 1
[Server] Sent to Client (Queue 9): FROM CLIENT 1
[Server] Recv from Client (Queue 10): From client 2
[Server] Sent to Client (Queue 10): FROM CLIENT 2
```

```
[Client (Queue 9)] Send: From client 1
[Client (Queue 9)] Recv: FROM CLIENT 1
```

```
[Client (Queue 10)] Send: From client 2
[Client (Queue 10)] Recv: FROM CLIENT 2
```

Server: Creates a dedicated queue to receive all incoming client requests.

Clients: Each client creates its own private queue to receive responses from the server.

Pros: Capable of handling high-volume data transfers without queue congestion or blocking issues.

5. System V Semaphores

Used to synchronize behavior between processes.

Represents an integer managed by the Kernel (>0).

5.1. semget() Function – Creating and Opening a Semaphore Set

```
#include <sys/types.h>
#include <sys/sem.h>

int semget(key_t key, int nsems, int semflg);
/* Returns: semid (identifier of the semaphore set) on success; -1 on failure/
```

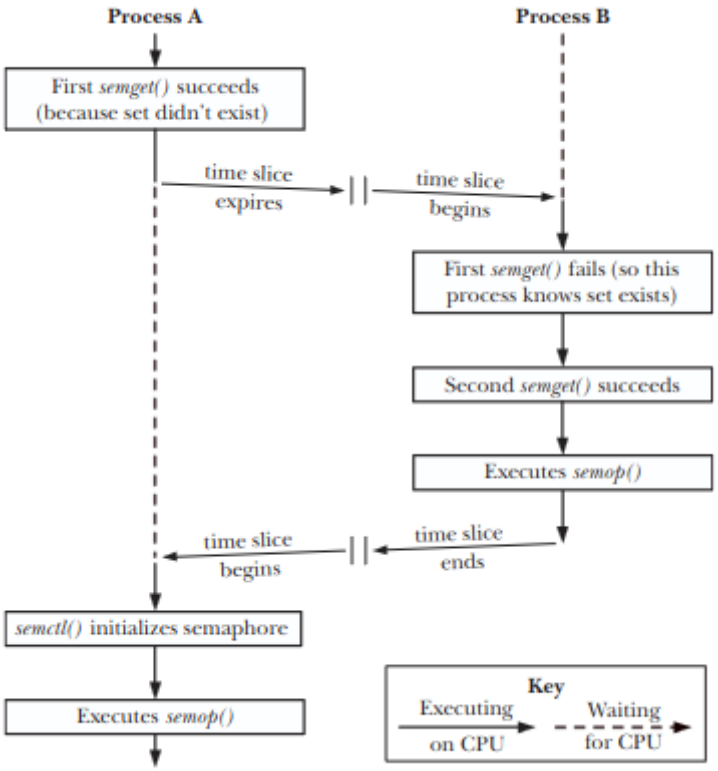
- key_t key: Identifier key used by processes to locate the semaphore set.
- int nsems: The number of semaphores in the set. Once created, this value cannot be changed.
- int semflg: Access permissions and control flags.

5.2. semctl() Function

The semid_ds structure inside the kernel:

```
struct semid_ds {
    struct ipc_perm sem_perm;  /* Ownership (UID, GID) and access permissions (mode) */
    time_t          sem_otime; /* Time of the last successful semop() call */
    time_t          sem_ctime; /* Time of creation or last property modification */
    unsigned long   sem_nsems; /* Number of semaphores in the set (fixed) */
};
```

5.3. Race Condition



When Process A runs:

- Successfully creates the semaphore by calling semget().
- CPU time slice expires -> The OS preempts Process A and switches to Process B before A gets a chance to initialize the semaphore's value.

When Process B runs:

- Calls semget() the first time with IPC_CREAT | IPC_EXCL flags -> Fails with EEXIST (because A already created the semaphore).
- Calls semget() a second time (without IPC_EXCL) -> Succeeds in opening the semaphore set and receives the semid.
- Process B uses the semaphore created by A, but it hasn't been initialized yet -> B operates on uninitialized/garbage values.

Context switch back to Process A:

- Process A wakes up and calls semctl() to initialize the value -> A overwrites B's work.

```
#define KEY 0x1234
union semun
{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};
```

```
int main(void)
{
    int semid;
    key_t key = KEY;
    semid = semget(key, 1, IPC_CREAT | IPC_EXCL | 0666);
    if (semid != -1)
    {
        printf("[Process A] Created semid: %d\n", semid);
        sleep(6);
        union semun arg;
        arg.val = 1;
        if (semctl(semid, 0, SETVAL, arg) == -1)
        {
            perror("semctl SETVAL");
            exit(EXIT_FAILURE);
        }
        printf("[Process A] Set value = 1 done.\n");
    }
    else if (errno == EEXIST)
    {
        semid = semget(key, 1, 0666);
        if (semid == -1)
        {
            perror("semget");
            exit(EXIT_FAILURE);
        }
        printf("[Process B] Opened existing semid: %d\n", semid);
        int val = semctl(semid, 0, GETVAL);
        printf("[Process B] Current value: %d \n", val);
        struct sembuf sop = {0, -1, 0};
        printf("[Process B] Block\n");
        fflush(stdout);
        if (semop(semid, &sop, 1) == -1)
        {
            perror("semop");
            exit(EXIT_FAILURE);
        }
        printf("[Process B] Current value: %d\n", semctl(semid, 0, GETVAL));
    }
    else
```

```
    {
        perror("semget");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

Output:

Process A

```
[Process A] Created semid: 4
[Process A] Set value = 1 done.
```

Process B

```
[Process B] Opened existing semid: 4
[Process B] Current value: 0
[Process B] Block
[Process B] Current value: 0
```

6. POSIX Semaphore

A synchronization mechanism (for locking shared resources) applicable to both Processes and Threads.

A non-negative integer variable.

Advantages over System V Semaphores

Atomic Initialization: Eliminates race conditions during setup.

- In System V: Creating the semaphore (semget) and assigning its initial value (semctl with SETVAL) are separate operations -> leads to potential race conditions.
- In POSIX: Functions such as sem_open(name, O_CREAT, 0644, value) or sem_init(&sem, pshared, value) perform creation and initialization atomically in a single call -> completely avoids initialization race conditions.

Thread Support: Directly supports multithreading (pthread), which System V cannot natively do.

Higher Performance: Increments and decrements occur directly in User Space at CPU instruction speeds, eliminating kernel call overhead. The system only invokes the Kernel to put a process to sleep in its wait queue when a conflict or blocking condition occurs.

Fix Race Condition in System V Semaphore

```
#define SEM_NAME "/posix_race_fix"
int main(void)
{
    sem_t *sem = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0666, 1);
    if (sem != SEM_FAILED)
    {
        printf("[Process A] Creat Sem\n");
        sleep(6);
        printf("[Process A] Done.\n");
        sem_close(sem);
        sem_unlink(SEM_NAME);
    }
    else if (errno == EEXIST)
    {
        sem = sem_open(SEM_NAME, 0);
        if (sem == SEM_FAILED)
        {
            perror("sem_open");
            exit(EXIT_FAILURE);
        }
        int val;
        sem_getvalue(sem, &val);
        printf("[Process B] Current value: %d\n", val);
        if (sem_wait(sem) == -1)
        {
            perror("sem_wait");
            exit(EXIT_FAILURE);
        }
        sem_getvalue(sem, &val);
        printf("[Process B] Current value: %d\n", val);
        sem_post(sem);
        sem_close(sem);
    }
    else
    {
        perror("sem_open");
        exit(EXIT_FAILURE);
    }
```

```
    return 0;
}
```

Output

Process A

```
[Process A] Creat Sem
[Process A] Done.
```

Process B

```
[Process B] Current value: 1
[Process B] Current value: 0
```

Even if Process A sleeps for 6 seconds right after creation, the value of the semaphore on RAM is already 1.

When Process B opens and reads it, it immediately sees Current value: 1, calls sem_wait() to decrement from 1 to 0, and acquires the lock instantly

7. System V Shared Memory

Two or more processes share a physical memory region. This memory region is mapped directly into the user space of each process.

The kernel does not intervene in the data transfer process. Once Process A finishes writing, Process B sees the data on RAM immediately without needing system calls to copy data through the kernel (unlike pipes or message queues).

A synchronization mechanism is required to avoid race conditions and overwriting data.

7.1. APIs

shmget(): Initializes or retrieves a shared memory segment identifier.

shmat(): Maps the memory segment from RAM into the virtual address space, returning a memory pointer.

shmdt(): Detaches the shared memory segment, notifying the OS that the process has finished using it.

shmctl(): Removes the shared memory segment from the kernel (it is physically deleted only when no processes remain attached to it).

7.2. Location of Shared Memory in Virtual Memory

When calling shmat(), the kernel places the Shared Memory into the unmapped space between the Heap and Stack, typically starting around address 0x40000000.

```
#define SHM_SIZE 102400
int main(void)
{
    int shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid == -1)
    {
        perror("shmget");
        exit(EXIT_FAILURE);
    }
    printf(" shmid: %d\n", shmid);
    char *addr = (char *)shmat(shmid, NULL, 0);
    if (addr == (void *)-1)
    {
        perror("shmat");
        exit(EXIT_FAILURE);
    }
    printf(" Attached at: %p (PID: %d)\n", (void *)addr, getpid());
    strcpy(addr, "Hello Shared Memory!");
    printf(" Data: %s\n\n", addr);
    char cmd[128];
    snprintf(cmd, sizeof(cmd), "cat /proc/%d/maps | grep -E '(SYSV|\\[stack\\]|\\[heap\\])'",
getpid());
    system(cmd);
    shmdt(addr);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

Output

```
 shmid: 5
 Attached at: 0x740d4c891000 (PID: 37996)
 Data: Hello Shared Memory!

609cb624c000-609cb626d000 rw-p 00000000 00:00 0                          [heap]
```

```
740d4c891000-740d4c8aa000 rw-s 00000000 00:01 5                           /SYSV00000000
(deleted)
7ffca09bc000-7ffca09dd000 rw-p 00000000 00:00 0                           [stack]
```

Lowest address (0x069...): Heap memory region growing upwards.

Middle address (0x704...): Shared memory region.

High address (0x7ff...): Stack memory region growing downwards.

7.3. Shared Memory combined with Semaphores

```
#define LOOP_COUNT 100000
#define SEM_NAME "/counter_lock"
int main()
{
    sem_unlink(SEM_NAME);
    sem_t *sem = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0666, 1);
    if (sem == SEM_FAILED)
    {
        perror("sem_open");
        exit(1);
    }
    int shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
    if (shmid == -1)
    {
        perror("shmget");
        exit(1);
    }
    int *counter = (int *)shmat(shmid, NULL, 0);
    *counter = 0;
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        exit(1);
    }
    else if (pid == 0)
    {
        for (int i = 0; i < LOOP_COUNT; i++)
        {
```

```
            sem_wait(sem);
            (*counter)++;
            sem_post(sem);
        }
        shmdt(counter);
        sem_close(sem);
        exit(0);
    }
    else
    {
        for (int i = 0; i < LOOP_COUNT; i++)
        {
            sem_wait(sem);
            (*counter)++;
            sem_post(sem);
        }
        wait(NULL);
        printf("Result: %d \n", *counter);
        shmdt(counter);
        shmctl(shmid, IPC_RMID, NULL);
        sem_close(sem);
        sem_unlink(SEM_NAME);
    }
    return 0;
}
```

Before incrementing the counter variable, a process must call sem_wait() to acquire the lock.

Any other process that arrives later must wait.

Once the increment is complete, it calls sem_post() to release the lock, allowing the waiting process to enter.

Lab 2 : producer-consumer via shared memory + semaphore

```
#define NUM_ITEMS 5
#define SEM_EMPTY "/sem_empty"
#define SEM_FULL  "/sem_full"
int main()
{
    sem_unlink(SEM_EMPTY);
```

```
sem_unlink(SEM_FULL);
sem_t *sem_empty = sem_open(SEM_EMPTY, O_CREAT | O_EXCL, 0666, 1);
sem_t *sem_full  = sem_open(SEM_FULL,  O_CREAT | O_EXCL, 0666, 0);
if (sem_empty == SEM_FAILED || sem_full == SEM_FAILED)
{
    perror("sem_open");
    exit(1);
}
int shmid = shmget(IPC_PRIVATE, sizeof(int), IPC_CREAT | 0666);
if (shmid == -1)
{
    perror("shmget");
    exit(1);
}
int *buffer = (int *)shmat(shmid, NULL, 0);
pid_t pid = fork();
if (pid < 0)
{
    perror("fork");
    exit(1);
}
else if (pid == 0)
{
    for (int i = 1; i <= NUM_ITEMS; i++)
    {
        sem_wait(sem_full);
        int item = *buffer;
        printf("[Consumer] Recv item: %d\n", item);
        sem_post(sem_empty);
        sleep(1);
    }
    shmdt(buffer);
    sem_close(sem_empty);
    sem_close(sem_full);
    exit(0);
}
else
{
    for (int i = 1; i <= NUM_ITEMS; i++)
    {
```

```
            sem_wait(sem_empty);
            *buffer = i * 10;
            printf("[Producer] Made item: %d\n", *buffer);
            sem_post(sem_full);
            sleep(1);
        }
        wait(NULL);
        shmdt(buffer);
        shmctl(shmid, IPC_RMID, NULL);
        sem_close(sem_empty);
        sem_close(sem_full);
        sem_unlink(SEM_EMPTY);
        sem_unlink(SEM_FULL);
    }
    return 0;
}
```

SEM_EMPTY (Initialized to 1): Represents the number of available empty slots in the buffer. Initially, the buffer is empty, so its value is set to 1—allowing the Producer to proceed first.

SEM_FULL (Initialized to 0): Represents the number of items currently available in the buffer. Initially, there are no items, so its value is set to 0—forcing the Consumer to wait.

8. Signal

Lab 3

Registering Signal Handlers for SIGINT and SIGALRM

- SIGINT: Used to clean up IPC resources before exiting, preventing resource leaks in the Linux kernel.
- SIGALRM: Typically paired with the alarm(seconds) system call to implement a Timeout mechanism for blocking IPC syscalls

Using strace for Observation:

- strace monitors the system calls that a program issues to the Linux kernel.
- Can filter for IPC-related syscalls and Signal-related syscalls.

Send Signal

```
#define QUEUE_KEY 0x9999
struct msg_buffer
{
    long mtype;
    char mtext[128];
};
int main(int argc, char *argv[])
{
    int qid = msgget(QUEUE_KEY, 0666);
    if (qid == -1)
    {
        perror("msgget - queue might not exist");
        exit(1);
    }
    struct msg_buffer msg;
    msg.mtype = 1;
    if (argc > 1)
    {
        strncpy(msg.mtext, argv[1], sizeof(msg.mtext) - 1);
    }
    else
    {
        strcpy(msg.mtext, "Hello from Sender!");
    }
    msg.mtext[sizeof(msg.mtext) - 1] = '\0';
    if (msgsnd(qid, &msg, strlen(msg.mtext) + 1, 0) == -1)
    {
        perror("msgsnd");
        exit(1);
    }
    printf("[Sender] Sent message: \"%s\" to Queue ID %d\n", msg.mtext, qid);
    return 0;
}
```

Handler Signal

```
#define QUEUE_KEY 0x9999
struct msg_buffer
{
    long mtype;
```

```
    char mtext[128];
};
int g_qid = -1;
void cleanup_and_exit(const char *reason, int exit_code)
{
    if (g_qid != -1)
    {
        if (msgctl(g_qid, IPC_RMID, NULL) == -1)
        {
            perror("msgctl");
        }
        else
        {
            printf("Deleted queue ID: %d\n", g_qid);
        }
        g_qid = -1;
    }
    printf("Exit: %s\n", reason);
    exit(exit_code);
}
void handle_signal(int signum)
{
    if (signum == SIGINT)
    {
        printf("\nCaught SIGINT\n");
        cleanup_and_exit("Interrupted", 0);
    }
    else if (signum == SIGALRM)
    {
        printf("\nCaught SIGALRM\n");
        cleanup_and_exit("Timeout", 1);
    }
}
int main()
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
```

```
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        exit(1);
    }
    if (sigaction(SIGALRM, &sa, NULL) == -1)
    {
        perror("sigaction SIGALRM");
        exit(1);
    }
    g_qid = msgget(QUEUE_KEY, IPC_CREAT | 0666);
    if (g_qid == -1)
    {
        perror("msgget");
        exit(1);
    }
    printf("Queue created ID: %d\n", g_qid);
    printf("Alarm set: 5s\n");
    alarm(5);
    struct msg_buffer msg;
    printf("Waiting for message...\n");
    ssize_t bytes = msgrcv(g_qid, &msg, sizeof(msg.mtext), 1, 0);
    if (bytes == -1)
    {
        if (errno != EINTR)
        {
            perror("msgrcv");
        }
    }
    else
    {
        alarm(0);
        printf("Received: %s\n", msg.mtext);
        cleanup_and_exit("Success", 0);
    }
    return 0;
}
```

Output:

```
rt_sigaction(SIGINT, {sa_handler=0x5c2cf835e35f, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0x72f611445330}, NULL, 8) = 0
rt_sigaction(SIGALRM, {sa_handler=0x5c2cf835e35f, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0x72f611445330}, NULL, 8) = 0
msgget(0x9999, IPC_CREAT|0666)          = 16
Queue created ID: 16
Alarm set: 5s
Waiting for message...
msgrcv(16, 0x7ffe1f453d30, 128, 1, 0)   = ? ERESTARTNOHAND (To be restarted if no handler)
--- SIGALRM {si_signo=SIGALRM, si_code=SI_KERNEL} ---

Caught SIGALRM
msgctl(16, IPC_RMID, NULL)              = 0
Deleted queue ID: 16
Exit: Timeout
+++ exited with 1 +++
```

8.1. Why sigaction() is "Reliable" vs. Old signal()?

Persistent Disposition

- signal() (Unreliable): In the legacy System V signal model, as soon as a signal was delivered and its handler began execution, the kernel automatically reset the signal's disposition back to default (SIG_DFL). To catch subsequent signals, developers had to explicitly call signal() again from within the handler itself.
- sigaction() (Reliable): The registered handler remains persistently installed until explicitly changed. The kernel never automatically resets it to SIG_DFL.

Automatic Signal Masking

- sigaction() automatically adds the signal currently being handled to the process signal mask (sa_mask) during handler execution. This prevents recursive, back-to-back signal deliveries from corrupting program state.

8.2. Race Condition in Unreliable Signals

In Unreliable Signals

```
void unreliable_handler(int sig)
{
    (void)sig;
    printf("[Child] Handling signal 1...\n");
```

```
    usleep(50000);
    printf("[Child] Done handling.\n");
}
int main(void)
{
    pid_t pid = fork();
    if (pid == 0)
    {
        struct sigaction sa;
        sa.sa_handler = unreliable_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = SA_RESETHAND | SA_NODEFER;
        sigaction(SIGUSR1, &sa, NULL);
        printf("[Child] Waiting for signals...\n");
        while (1) pause();
    }
    else
    {
        sleep(1);
        printf("[Parent] Send signal 1\n");
        kill(pid, SIGUSR1);
        usleep(10000);
        printf("[Parent] Send signal 2\n");
        kill(pid, SIGUSR1);
        int status;
        waitpid(pid, &status, 0);
        if (WIFSIGNALED(status))
        {
            printf("[Parent] Child was KILLED by signal %d!\n", WTERMSIG(status));
        }
    }
    return 0;
}
```

Output

```
[Child] Waiting for signals...
[Parent] Send signal 1
[Child] Handling signal 1...
[Parent] Send signal 2
```

```
[Parent] Child was KILLED by signal 10!
```

The 2nd signal arrived too quickly (after 10ms) while the handler for the 1st signal was still executing (requiring 50ms).

Because the OS had already reset the disposition to default, the Operating System misinterpreted the signal and killed the process

Fixed

```
volatile int count = 0;

void reliable_handler(int sig)
{
    (void)sig;
    count++;
    printf("[Child] Handling signal %d...\n", count);
    usleep(50000);
    printf("[Child] Done signal %d.\n", count);
}

int main(void)
{
    pid_t pid = fork();

    if (pid == 0)
    {
        struct sigaction sa;
        sa.sa_handler = reliable_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = 0;
        sigaction(SIGUSR1, &sa, NULL);

        printf("[Child] Waiting for signals...\n");
        while (count < 2) pause();

        printf("[Child] Done all signals, exit safely.\n");
        exit(0);
    }
    else
    {
```

```
        sleep(1);

        printf("[Parent] Send signal 1\n");
        kill(pid, SIGUSR1);

        usleep(10000);
        printf("[Parent] Send signal 2\n");
        kill(pid, SIGUSR1);

        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status))
        {
            printf("[Parent] Child exited safely (code %d).\n", WEXITSTATUS(status));
        }
    }
    return 0;
}
```

Output

```
[Child] Waiting for signals...
[Parent] Send signal 1
[Child] Handling signal 1...
[Parent] Send signal 2
[Child] Done signal 1.
[Child] Handling signal 2...
[Child] Done signal 2.
[Child] Done all signals, exit safely.
[Parent] Child exited safely (code 0).
```

sigaction() with default configurations

- Persistent Disposition: The handler is retained continuously → Prevents the OS from misinterpreting subsequent signals and unexpectedly terminating the process.
- Automatic Masking & Pending State: Duplicate signals arriving during handler execution are automatically blocked (sa_mask) and marked as pending → Guarantees safe and uninterruptible execution of the current handler without state corruption.

8.3. Example

```
volatile sig_atomic_t got_signal = 0;

void handler(int sig)
{
    (void)sig;
    got_signal = 1;
    printf("   [Handler] Caught SIGINT!\n");
}

int main(void)
{
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    sigset_t block_mask, prev_mask, pending_mask;

    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGINT);

    printf("[Main] Blocking SIGINT...\n");
    sigprocmask(SIG_BLOCK, &block_mask, &prev_mask);

    printf("[Main] Entering Critical Section...\n");
    printf("[Main] Self-sending SIGINT via raise()...\n");
    raise(SIGINT);

    sigpending(&pending_mask);
    if (sigismember(&pending_mask, SIGINT))
    {
        printf("[Main] Verified: SIGINT is currently PENDING (blocked safely).\n");
    }

    printf("[Main] Leaving Critical Section.\n");

    printf("[Main] Calling sigsuspend() to atomically unblock & wait...\n");
    while (!got_signal)
    {
```

```
        sigsuspend(&prev_mask);
    }

    printf("[Main] Restoring original mask...\n");
    sigprocmask(SIG_SETMASK, &prev_mask, NULL);

    printf("[Main] Done safely!\n");
    return 0;
}
```

- To prevent interruptions while the process is updating critical data. sigprocmask holds the signal in a Pending state until the execution of the critical section completes.
- raise(SIGINT): Sends a SIGINT signal to the current process itself. Since SIGINT is currently blocked, the process is not interrupted; instead, the kernel marks the signal as pending.
- sigpending(): Inspects the set of pending signals signals that have been raised but are currently blocked.
- sigsuspend(): Atomically combines unblocking the signal and putting the process to sleep to wait for a signal. This eliminates any race condition gap where an interrupt could occur between unblocking and sleeping.

Revision #7
Created 2026-09-15 01:48:51 UTC by Huyen Nguyen
Updated 2026-09-21 06:38:18 UTC by Huyen Nguyen